

SE 103 Week 5 — Shell Practice Submitted Through GitHub (Conversation with Copilot)

Student Name: Rana AlSanousi

GitHub username: RanaAlSanousi

Date Submitted: 05.10.2026

Copilot conversation:



The screenshot shows a terminal window titled 'Ubuntu' with a dark theme. At the top, there are tabs for 'Current', 'Sessions', 'Issues', 'Pull requests', and 'Gists'. The main chat area displays a conversation. A system message at the top states: '! Authentication is required to use the available model catalog; omitting model ID completions'. Below this, two status messages are shown: '• GitHub MCP Server: Connected' and '• MCP Servers reloaded: 1 server connected'. The user's prompt is highlighted in a teal box: '> I'm working in a local Git repo on WSL (Ubuntu), on branch main. I made two commits locally that I have NOT pushed yet, and I want to combine them into a single commit with a new message: "combine local commits into one". Give me the exact command(s) to do this safely without losing any of my changes, explain what each command does, and warn me what could go wrong if I had already pushed those commits.' The Copilot response is shown below the prompt, starting with '> Thought for 6s' and a bullet point: '• From the repository root, first check that the working tree is clean:'. It then lists a series of commands: `git status --short`, `git log -2 --oneline`, `git branch backup-before-squash`, `git reset --soft HEAD~2`, `git commit -m "combine local commits into one"`, and `git status`. A warning follows: 'If this prints any files, stop before continuing; the commands below assume there are no uncommitted changes, so you don't accidentally include them in the new commit. Then run:'. The terminal status bar at the bottom shows the path '~ / projects / SE103-Week5 [≠main]' and 'Session: 0.23 AIC used'.

```
! Authentication is required to use the available model catalog; omitting model ID completions

• GitHub MCP Server: Connected

• MCP Servers reloaded: 1 server connected

> I'm working in a local Git repo on WSL (Ubuntu), on branch main. I made two commits locally that I have NOT pushed yet, and I want to combine them into a single commit with a new message: "combine local commits into one". Give me the exact command(s) to do this safely without losing any of my changes, explain what each command does, and warn me what could go wrong if I had already pushed those commits.

> Thought for 6s

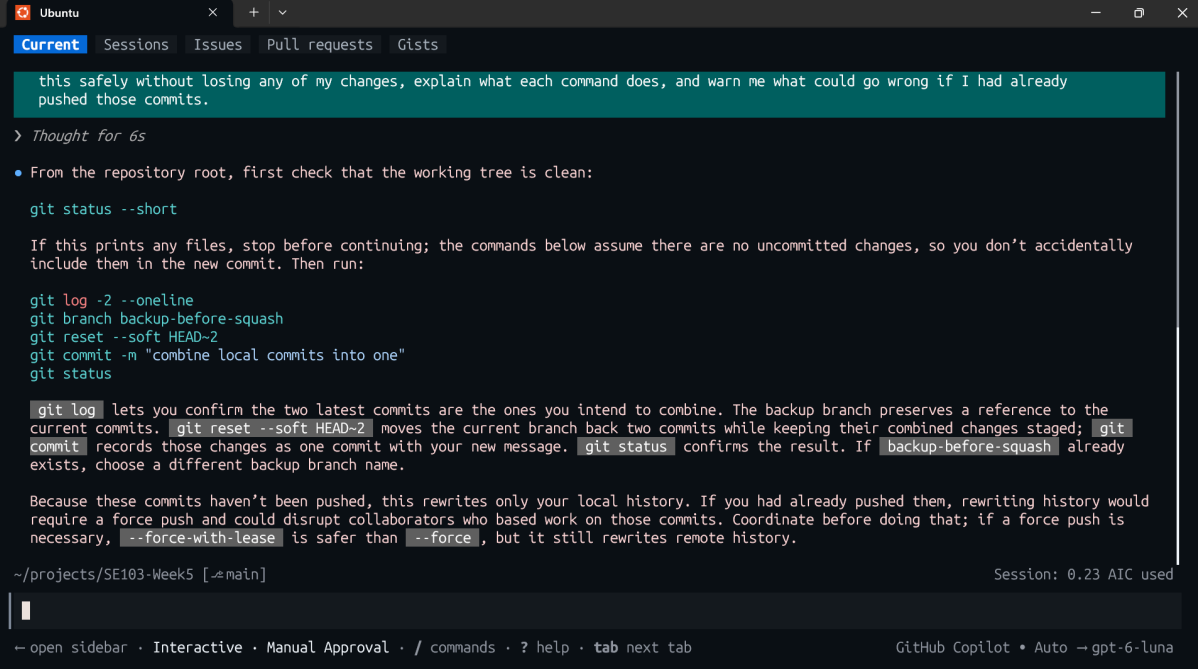
• From the repository root, first check that the working tree is clean:

git status --short

If this prints any files, stop before continuing; the commands below assume there are no uncommitted changes, so you don't accidentally include them in the new commit. Then run:

git log -2 --oneline
git branch backup-before-squash
git reset --soft HEAD~2
git commit -m "combine local commits into one"
git status

~/projects/SE103-Week5 [≠main] Session: 0.23 AIC used
```



This screenshot is a continuation of the previous one, showing the same terminal window. The user's prompt is repeated at the top. The Copilot response continues with the same list of commands and the warning. It then provides a detailed explanation: '`git log` lets you confirm the two latest commits are the ones you intend to combine. The backup branch preserves a reference to the current commits. `git reset --soft HEAD~2` moves the current branch back two commits while keeping their combined changes staged; `git commit` records those changes as one commit with your new message. `git status` confirms the result. If `backup-before-squash` already exists, choose a different backup branch name.' It concludes with another warning: 'Because these commits haven't been pushed, this rewrites only your local history. If you had already pushed them, rewriting history would require a force push and could disrupt collaborators who based work on those commits. Coordinate before doing that; if a force push is necessary, `--force-with-lease` is safer than `--force`, but it still rewrites remote history.' The terminal status bar remains the same.

```
this safely without losing any of my changes, explain what each command does, and warn me what could go wrong if I had already pushed those commits.

> Thought for 6s

• From the repository root, first check that the working tree is clean:

git status --short

If this prints any files, stop before continuing; the commands below assume there are no uncommitted changes, so you don't accidentally include them in the new commit. Then run:

git log -2 --oneline
git branch backup-before-squash
git reset --soft HEAD~2
git commit -m "combine local commits into one"
git status

git log lets you confirm the two latest commits are the ones you intend to combine. The backup branch preserves a reference to the current commits. git reset --soft HEAD~2 moves the current branch back two commits while keeping their combined changes staged; git commit records those changes as one commit with your new message. git status confirms the result. If backup-before-squash already exists, choose a different backup branch name.

Because these commits haven't been pushed, this rewrites only your local history. If you had already pushed them, rewriting history would require a force push and could disrupt collaborators who based work on those commits. Coordinate before doing that; if a force push is necessary, --force-with-lease is safer than --force, but it still rewrites remote history.

~/projects/SE103-Week5 [≠main] Session: 0.23 AIC used
```

Myself (prompt): I'm working in a local Git repo on WSL (Ubuntu), on branch main. I made two commits locally that I have NOT pushed yet, and I want to combine them into a single commit with a new message:

"combine local commits into one". Give me the exact command(s) to do this safely without losing any of my changes, explain what each command does, and warn me what could go wrong if I had already pushed those commits.

Copilot's reply: Thought for 6s

From the repository root, first check that the working tree is clean:

```
git status --short
```

If this prints any files, stop before continuing; the commands below assume there are no uncommitted changes, so you don't accidentally include them in the new commit. Then run:

```
git log -2 --oneline
git branch backup-before-squash
git reset --soft HEAD~2
git commit -m "combine local commits into one"
git status
```

`git log` lets you confirm the two latest commits are the ones you intend to combine. The backup branch preserves a reference to the current commits. `git reset --soft HEAD~2` moves the current branch back two commits while keeping their combined changes staged; `git commit` records those changes as one commit with your new message. `git status` confirms the result. If `backup-before-squash` already exists, choose a different backup branch name.

Because these commits haven't been pushed, this rewrites only your local history. If you had already pushed them, rewriting history would require a force push and could disrupt collaborators who based work on those commits. Coordinate before doing that; if a force push is necessary, `--force-with-lease` is safer than `--force`, but it still rewrites remote history.

Performance Reflection:

Ranking: 8/10

Copilot gave a correct and safe solution. `git reset --soft HEAD~2` followed by `git commit -m "..."` combines the last two commits while keeping all the changes staged. It also added a backup branch before rewriting anything, a clean-working-tree check, and a warning that force-pushing would be needed if the commits had already been pushed. I took one

point off because it didn't mention `git rebase -i HEAD~2`, the other common way to squash, and it assumed a clean working tree.